Performance Test Results — Milestone Status

Document control

Forever Hotel • Manager Dashboard • Guide deliverable 9

Prepared 2026-10-25, Asia/Colombo. Source development, caef8822afc0c44b030bea2453e23eb01d4af42a. Approximate project progress: 80%, as stated by the owner. Performance execution status: Not tested.

1. Assessment

MAD contains explicit staging performance-runner, metric and evaluator source. No approved executed load-test output, event timing evidence or accepted staging environment was available for this snapshot. This report documents that result status, available tooling, limited prior observations and the remaining plan. It does not present proposed load profiles as measured success.

No load, stress, soak, live-service probes or security tests were executed while creating this document. The approximate 80% milestone includes built MAD features and test tooling; measured performance and acceptance remain unfinished work.

2. Requirement and acceptance matrix

Requirement	Original target	Evidence state
NFR-01	95% of API requests within 500 ms at 100 concurrent users under normal load	Not measured in a controlled workload
NFR-02	Cross-service events propagated within one second	No synchronized producer/broker/consumer timings attached
NFR-03	Orders/tasks/room updates visible within two seconds	No correlated producer-to-browser timing report
NFR-04	99.5% monthly availability excluding scheduled maintenance	No month-long monitoring record
NFR-14	At least 300 concurrent users without performance degradation	Not measured; socket process-local constraints require review
NFR-15 / NFR-21	Peak food-order acceptance and KOT printing targets	Owned by KMS/FOSS/printing, not established by a read-only MAD load run

The implementation's proposed 300-user comparison permits no more than a 10% p95 increase over the 100-user baseline. This is a proposed engineering criterion, not an approved replacement for the SRS phrase “without performance degradation.” The requirement owner must decide how the SRS is evaluated before execution.

3. Tooling present in the implementation

Source	Responsibility
backend/test/performance/run.ts	Approved-staging-only API/socket workload, 100/300-user profiles, output collection
backend/test/performance/metrics.ts	Percentiles and timing-evidence evaluation logic
backend/test/performance/evaluate.ts	Baseline/capacity comparison and timing evaluation; retains owner/environment acceptance as pending
backend/test/performance/ metrics.spec.ts	Unit-test source for evaluation behavior; not a load-test result

The runner requires explicit staging approval, safe origin URLs, distinct active manager sessions and a unique run label. Session inputs are private and must never be appended to the report. The source includes a five-minute warm-up and fifteen- minute measurement phase, one persistent socket per user, and scheduled API reads. Tool presence does not establish successful compilation or execution in this report.

4. Proposed scope for the unfinished measurement phase

The read workload spans booking/revenue/food/service/worker analytics, operational snapshots, notifications, staff, complaints and promotions. It does not mutate bookings, send emails, claim tasks or process payments. Provider write throughput therefore needs its own coordinated workload.

Run	Users	Warm-up	Measured interval	Gate / observation
Baseline	100	5 minutes	15 minutes	API latency distribution, errors, socket stability, per-route statistics
Capacity	300	5 minutes	15 minutes	Same measurements; compare p95 with reviewed baseline criterion
Event timing	Owner-approved producer activity	Clock verification first	Agreed observation window	Producer commit/ publish, MAD receipt/ dispatch and browser render correlation

Run	Users	Warm-up	Measured interval	Gate / observation
Recovery	Separate controlled rehearsal	Stable baseline	Owner-defined	Broker/socket disconnect, reconnect snapshots, pending durable work

These profiles are not approved execution records. Run only after source-owner coordination, infrastructure authorization and a representative synthetic dataset. Do not execute the load runner against an unapproved Neon branch or production.

5. Result ledger

Result field	100-user baseline	300-user capacity
Approved environment and owner	Not supplied	Not supplied
Execution date / tested commit	Not tested	Not tested
Completed request count	Not measured	Not measured
p50 / p95 / p99 latency	Not measured	Not measured
Unexpected HTTP error rate	Not measured	Not measured
Socket disconnect/error count	Not measured	Not measured
Event propagation / UI latency	Not measured	Not measured
CPU / memory / DB pool saturation	Not measured	Not measured
Result artifact / CI link	Not available	Not available
SRS acceptance	Pending	Pending

6. Existing browsing observations — not a benchmark

The previously supplied localhost.har contained 83 captured requests and six 503 responses: occupancy and low-booking requests around 3.5 seconds, and four sign-in attempts around 10–11 seconds. A later sign-in succeeded. Multiple socket connections upgraded successfully with HTTP 101. These observations came from the earlier HAR review; they are not a new measurement on 2026-10-25.

This is an uncontrolled local browsing sample. It has no defined user concurrency, warm-up, representative traffic mix, database state or stable test window. WebSocket lifetime in a HAR is not message latency. Cache entries and repeated navigation further prevent treating it as a controlled p95 result. The six errors must not be extrapolated into a whole-system availability percentage.

The raw HAR is not embedded because it may contain credentials, sessions or personal data. Preserve only a sanitized copy in any evidence handoff.

7. Performance risks and investigation priorities

1. Correlate intermittent login/analytics 503s across BFF, gateway, backend and DB timing. Do not assume the root cause is Neon cold start, pooling or gateway timeout without evidence; those are investigation hypotheses only.

2. Supply dated availability/maintenance for all real rooms. Missing history can produce an intentional unavailable response rather than a slow successful query.

3. Check query plans and indexes using approved staging data; source SQL presence does not prove plan efficiency at hotel scale.

4. Review local rate limits before a representative load run. Do not disable security controls merely to conceal 429s or alter acceptance metrics afterward.

5. Distinguish direct socket traffic from REST gateway traffic. Process-local tickets and fanout mean horizontal scaling is not automatically correct.

6. Measure provider event versions, durability and UI reloading under burst/reconnect conditions, without equating broker acknowledgement with browser visibility.

8. Required execution evidence and release decision

Before running: capture the tested commit/image identity, synthetic dataset size, database branch class, service versions, instance/CPU/memory sizes, network region, pool settings, rate-limit settings, producer clocks and approved workload. Record values that affect results but redact credentials and private network endpoints.

After running: retain raw non-secret samples, per-route latency/error summaries, 100/300-user results, correlated event timings, resource telemetry, evaluator output and a reviewer decision. Repeated runs should preserve run labels and comparability. The evaluator explicitly leaves owner/environment acceptance pending even when its numerical checks pass.

Decision for this milestone: report prepared; performance targets not proven. Measured performance, recovery and owner acceptance remain in the unfinished work. This PDF is not a passing performance certificate and does not authorize production.

References

SRS NFR-01/02/03/04/14; guide sections 6, 10.1 and 10.4; backend/test/performance/; documents/testing/test-register.md; source-based architecture analysis in docs/reports/mad-built-system-and-design-comparison.md.